

Binary search

time: $O(\log n)$, space: $O(1)$ | Encontrar un valor en un array ordenado.

```
#include <vector>
using namespace std;
int binary_search_idx(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Fast I/O (C++)

```
// Fast I/O -- put these two lines at the very start of main():
ios::sync_with_stdio(false);
cin.tie(nullptr);
// Then read/write as usual (now unbuffered from C stdio, much faster):
//   int z; cin >> z;           // input
//   cout << z << "\n";       // output ('\\n' is faster than endl)
```

Fast I/O (Python)

```
import sys

data = sys.stdin.buffer.read().split() # fast input: read all tokens at once
it = iter(data)
def read_int():                        # next integer from the input
    return int(next(it))

out = sys.stdout.write                 # fast output: out(str(z) + "\\n")
```

Dijkstra

time: $O((V+E) \log V)$, space: $O(V+E)$ | Camino más corto desde un origen con pesos no negativos.

```
#include <vector>
#include <queue>
#include <functional>
using namespace std;
const long long INF = 1e18;
vector<long long> dijkstra(int n, const vector<vector<pair<int, int>>>& adj, int src) {
    vector<long long> dist(n, INF);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Fenwick Tree (BIT)

build: $O(n)$, update: $O(\log n)$, query: $O(\log n)$, space: $O(n)$ | Sumas de prefijos con actualizaciones puntuales.

```
#include <vector>
using namespace std;
struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree(int n) : n(n), tree(n) {}
    void update(int index, int delta) {
        for (index++; index <= n; index += index & -index)
            tree[index - 1] += delta;
    }
    int query(int index) {
        int acc = 0;
```

```
        for (; index > 0; index -= index & -index)
            acc += tree[index - 1];
        return acc;
    }
    int query_range(int l, int r) { return query(r) - query(l); }
};
```

Convex hull

time: $O(n \log n)$, space: $O(n)$ | El menor polígono convexo que contiene un conjunto de puntos.

```
#include <vector>
#include <algorithm>
using namespace std;
struct P {
    long long x, y;
    bool operator<(const P& o) const {
        return x < o.x || (x == o.x && y < o.y);
    }
};
long long cross(const P& O, const P& A, const P& B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}
vector<P> convex_hull(vector<P> pts) {
    int n = pts.size();
    if (n < 3) return pts;
    sort(pts.begin(), pts.end());
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```